Security Review

A multi-agent red-team tried to break into student data. Here's what held, what didn't, and what got fixed.

Date	October 11, 2026
What was tested	The live backend on a throwaway copy of the database: row-level security, the server functions (groups, join codes, recurring tasks), the two AI functions, the web bundle, the repo, and the deploy pipeline.
How	AI agents acted as real student accounts and attacked in parallel. Every reported break-in had to be reproduced by a separate agent before it counted. After each round of fixes, the fixes themselves were attacked again, three rounds in all.
Headline	**No way in to another student's private data, and no account or group takeover.** Seven smaller issues turned up across three rounds: five in the original code and two in the first round of fixes. Six are fixed and covered by tests; one was accepted on purpose. The final round found nothing.

> **Bottom line for judges:** The things that matter most — one student can't read another's tasks, grades, goals, or group data, and can't delete or hijack anyone's work — were attacked directly and held. The issues found were about abuse limits and timestamp tampering, not stolen data. Every real one is fixed.

1. What the attackers could NOT do

These were attacked directly, as a non-member and as an outsider, and every attempt was correctly blocked (an access error or an empty result):

- Read another student's personal tasks, class list, or check-off history.
- Read a non-groupmate's private profile — goals, struggles, grade, school level, email. Groupmates see only a name.
- See a group's shared tasks, roster, or activity feed without being a member (or after leaving).
- Edit, move, or delete another student's task; change who created a task; forge the "created by" field.
- Make themselves owner of a group, remove the real owner, remove a member, or rotate a code without being the owner.
- Read the internal tables (join-attempt log, AI-usage counter) at all.
- Call the AI functions without being signed in, or read another student's data through them.

- Find any secret in the shipped website (the Claude key and the database admin key are server-only; only the public, by-design values appear).

2. What was found and fixed

Five issues survived independent verification in round 1. None exposed private data or allowed a takeover; they were abuse-limit and data-integrity gaps. Four are fixed, each with an automated test that proves the hole is closed; the fifth was accepted on purpose.

#	Severity	Issue	Fix	Status
1	MEDIUM	**Join-code guessing limit was beatable.** The "20 wrong tries a day" limit could be raced with many simultaneous requests, and making new free accounts gave fresh budgets.	The limit now locks per student so simultaneous tries can't slip past it. (Making new accounts is a sign-up setting — see §5.)	**Fixed + test**
2	MEDIUM	**AI spending wasn't capped overall.** The 40-calls-a-day limit was per student, so many free accounts could run up the Claude bill.	Added a whole-app daily ceiling on AI calls as a spending backstop. The usage counter is server-only, so students can't touch it, and each student counts only up to their own limit. Your Console spend cap stays as the final backstop.	**Fixed + test**
3	MEDIUM	**Upload-size limit was bypassable.** The 8 MB cap on AI requests trusted a header that an attacker controls, so a large upload could slip through.	The size is now counted on the actual data as it arrives and cut off past 8 MB.	**Fixed + test**
4	LOW	**Timestamps on shared tasks could be forged.** A member could backdate a shared task (changing how it's ranked for everyone) or set an arbitrary internal id.	The server stamps a shared task's creation time, both when it's created in a group and when it's moved into one. The app can no longer set the internal id or "last updated" at all.	**Fixed + test**

5	LOW	**AI functions allow any website to call them.** Acceptable because the only key is each student's login token (a browser can't attach someone else's), but worth recording.	Kept open (required for the phone apps), documented, and confirmed it never allows credentialed cross-site calls.	Accepted, documented

3. Attacking the fixes

A fix can open a new hole, so every fix was attacked again, round after round, until a round found nothing new.

Round	What it found	Outcome
2	**Two holes in the round-1 fixes.** (1) HIGH The new whole-app AI ceiling could be abused: a student could bump the usage counter directly, push it over the ceiling, and turn AI off for everyone that day. (2) LOW The shared-task timestamp fix could be bypassed by backdating a personal task, then moving it into a group.	**Both fixed + tests**
2	Confirmed holding: the join-code limit stayed at exactly 20 under 80 simultaneous guesses; the 8 MB upload cap stopped a 9 MB disguised upload; an outsider still read nothing across all 11 tables.	**Held**
3	**89 attack attempts, none got through.** These covered the round-2 fixes and a full sweep of the server functions and groups. The attackers tried using another student's AI quota, forged logins, 80 simultaneous requests, sneaking old timestamps in through moves, upserts and recurring tasks, taking over groups, reading private profiles, rejoining after being removed, SQL injection, and calling internal helper functions. Leftover default permissions on two internal counter tables were also removed. They weren't exploitable, but removing them adds a layer of safety.	**Clean: stopped here**

4. Two claims that did NOT hold up

The verification step rejected two reported issues — a reminder that the second-opinion pass matters:

- **A vulnerable sub-library in the web bundle.** The library is genuinely outdated, but no path in the app actually reaches the vulnerable code, so it isn't exploitable here. Noted for a future dependency bump.
- **Shared browser storage on GitHub Pages.** Every project site under `ashih-10.github.io` shares one origin, so in theory they can read each other's browser storage. In the demo nothing private is at stake, and the real app should run on Vercel or a custom domain — which the README already says. Not a live vulnerability.

5. What's still in your hands (deployment settings)

Two of the issues above have a code fix *and* a settings side that only you can turn on, once the competition demo is done:

> - **Turn email confirmation back on** after the demo (Supabase → Authentication → Providers → Email), **but connect a real email provider first**. The setup guide has you turn it *off* so demo accounts work instantly; while it's off, anyone can make unlimited accounts, which is what weakens the join-code and AI-spend limits. Careful: Supabase's built-in email only delivers to members of your Supabase team (2 emails an hour) and isn't meant for real users. Without a provider, real students would never get their confirmation email. Set up Resend (free tier) under Authentication → SMTP Settings first, then turn confirmation on.
> - **Keep the Claude spend cap** you set in the Console (about $10). That's the final backstop on cost no matter what.

6. How to re-run this yourself

The security tests live in the repo and run against a local copy of the database in Docker — no real data, no cost:

```
npm test                # 58 logic tests (pepper formula, recurrence, demo backend)
npx supabase start      # local database in Docker
npm run test:db         # 27 attack tests (the 7 AI-function tests also need the fake Claude
server; see README)
```

The attack tests sign in as several students and try every break-in on this page, including the holes found in rounds 1 and 2. They fail if any hole reopens.